

MLPERF STORAGE · AI SSD EVALUATION

实际测试 + 模拟 Trace 测试 · 从 72 Case 提炼的可执行子集

|     |            |                                     |                                     |
|-----|------------|-------------------------------------|-------------------------------------|
| 1.0 | 2026-08-02 | 实际测试<br>A ×9 /<br>Trace 模<br>拟 B ×6 | docs/AI_SSD_TEST_PLAN_SIMPLIFIED.md |
|-----|------------|-------------------------------------|-------------------------------------|

基于 **AI\_SSD\_TEST\_PLAN.md** (72 Case 详细版) 提炼, 覆盖 Training / Checkpointing / KV Cache / VectorDB 四个家族。每条轨道都要求证明**实际命中 DUT**, 每个 Case 输出 **Pass / Fail / Invalid** 三态判定。

|                                                    |                                                         |
|----------------------------------------------------|---------------------------------------------------------|
| 0. 简化原则                                            | AI-CKP-002)                                             |
| 1. 已核实的仓库环境现状                                      | A5–A7. KV MLPerf Option 1/2/3<br>(对应 AI-KV-001/002/003) |
| 2. 测试轨道总览                                          | A8. VectorDB: 1M×1536 HNSW<br>(对应 AI-VDB-002)           |
| 3. 轨道 A: 实际测试 (9 个 Case)                           | A9. VectorDB: 1M×1536<br>DISKANN (对应 AI-VDB-003)        |
| A1. BASE 建档与路径检查 (对应 AI-BASE-001)                  |                                                         |
| A2. Training 大文件读: UNet3D/A100 (对应 AI-TRN-001)     | 4. 轨道 B: 模拟 Trace 测试 (6 个 Case)                         |
| A3. Training 小文件读: RetinaNet/B200 (对应 AI-TRN-004)  | B1. KV trace 生成: MLPerf Option 3 大对象 (对应 AI-KV-021 前半)  |
| A4. Checkpoint 突发写/读: Llama3-70B subset 8-rank (对应 | B2. KV Tier-2 过滤 + Direct replay<br>(对应 AI-KV-021)      |

B3. KV prefill / decode 分离 trace  
B6. 大容量模型 trace 模拟 (对应  
A-KV-002 / 无法跑的大模型)

# AI SSD 简化测试方案 (实际测试 + 模拟 Trace 测试)

B4. TP sweep 模拟 per-rank 对象大小 (对应 AI-KV-014 的 trace 形式)  
B5. VDB trace replay (对应 AI-VDB-015)  
5. 判定规则 (统一)  
6. 监控 (1 s 采样)  
7. 结果归档  
8. 执行顺序建议  
9. Definition of Pass (简化版)

版本: 1.0  
日期: 2026-08-02  
上游: docs/AI\_SSD\_TEST\_PLAN.md (72 Case 详细版) —— 本文件是从中提炼的**可立即执行子集**  
定位: 覆盖 Training / Checkpointing / KV Cache / VectorDB 四个家族, 每族选 1-3 个代表性 Case, 分「实际测试」与「模拟 trace 测试」两条轨道, 全部命令在本仓库当前 Windows 环境可直接运行或仅需一步前置安装。本文档配套仓库: MLPerf Storage Benchmark Suite v2.0.0b1 (Windows 环境)

## 0. 简化原则

- 从 72 个 Case 中按以下原则精选:
- 1. 每条轨道覆盖四个家族各至少一个 Case (大文件读 / 小文件读 / 突发写 / 小对象随机读 / 磁盘 ANN)。
  - 2. 优先选「当前主机可跑」的配置 (Now), Scaled / Experimental 一律不进入本轮。
  - 3. 容量敏感的大数据量 (983 GiB UNet3D/B200、1.38 TiB CosmoFlow 等) 用 trace 模拟 代替实跑: trace 反映对象大小与访问节奏, replay 施加真实 SSD 压力。
  - 4. 每个 Case 输出 Pass / Fail / Invalid 三态判定, Invalid (没命中 DUT、缓存污染、缺 rank) 不能靠性能好坏抵消。

## 1. 已核实的仓库环境现状

| 项                                     | 状态                                                                                                          | 影响                                                       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| mlpstorage.exe<br>CLI                 | 可用 (closed/open/whatif; training 模型 unet3d/resnet50/cosmoflow/dlrm/retinanet/flux; checkpointing run file)  | Training/Checkpoint 命令可用                                 |
| dlio 模块                               | 未安装                                                                                                         | Training 实际测试需先 pip install -e ".[full]" (含 DLIO+MPI 依赖) |
| kv_cache.cli<br>(kv-cache.py)         | 完整可用; 支持 --gpu-mem-gb/--cpu-mem-gb/--cache-dir/--io-trace-log/--prefill-only/--decode-only/-tensor-parallel | KV 实际测试与 trace 生成均可跑                                     |
| vdbbench-modular +<br>vdbbench.replay | 可用; pymilvus 3.0.1 已装; Milvus 三容器 (standalone/minio/etcd) healthy                                           | VDB 实际测试 + trace replay 可跑                               |

| 项                                   | 状态                                                                                          | 影响                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>--direct-io</code>            | <code>vdbbench.replay</code> 支持（绕过 Windows 页缓存）                                             | trace 模拟测试的冷读手段                                              |
| Windows<br><code>drop_caches</code> | 无                                                                                           | 冷态用 <code>--o-direct</code> / <code>--direct-io</code> 或重启实现 |
| 已有 smoke 证据                         | UNet3D AU 97.9%；KV Storage P95 433 ms；KV trace 455 行（Tier-2 58 行）；VDB 1K HNSW Recall@10=1.0 | 只作环境可运行证据，不作 golden                                          |

trace 文件格式（已确认）：`Timestamp,Operation,Object_Size_Bytes,Tier,Key,Phase`，`Tier-2` = NVMe 层。

## 2. 测试轨道总览

| 轨道         | 方法                                                                               | 真实 I/O 发生位置                                    | 适用                       |
|------------|----------------------------------------------------------------------------------|------------------------------------------------|--------------------------|
| A 实际测试     | workload 直接跑在 DUT 上                                                              | 应用自身（KV <code>cache-dir</code> 、VDB 数据卷、训练数据集） | 小规模实跑，验证链路 + 真实成绩        |
| B 模拟 trace | <code>--io-trace-log</code> 生成逻辑 trace → 过滤 → <code>replay --direct-io</code> 回放 | replay 工具                                      | 大模型 / 大容量 / 参数扫描，无法实跑时模拟 |

两条轨道都要证明实际命中 DUT：A 轨看应用指标 + PhysicalDisk 计数；B 轨看 replay 的 aligned bytes 与 P50/P95/P99。

## 3. 轨道 A：实际测试（9 个 Case）

### A1. BASE 建档与路径检查（对应 AI-BASE-001）

- **执行**：记录 4 块盘的型号/序列号/盘符/卷映射、空闲容量、温度、SMART 错误计数；确认 `data/cache/checkpoint/vdb` 卷 都在 DUT、结果目录在**非** DUT。
- **判定**：映射表齐全且与物理盘一致 → Pass；路径落错盘 → Invalid。

### A2. Training 大文件读：UNet3D/A100（对应 AI-TRN-001）

- **前置**：`pip install -e ".[full]"`（当前 `.venv` 缺 `dllo`）。
- **命令**：

```
mlpstorage whatif training unet3d datasize `
  --systemname ai-ssd-win --client-host-memory-in-gb 53 --num-client-hosts 1 `
  --max-accelerators 4 --accelerator-type a100

mlpstorage whatif training unet3d run file `
  --systemname ai-ssd-win --hosts 127.0.0.1 `
  --client-host-memory-in-gb 53 --num-client-hosts 1 `
  --num-accelerators 2 --accelerator-type a100 `
  --data-dir <DUT_DIR> --results-dir <RESULT_DIR> --o-direct
```

- **观测：**AU、samples/s、GiB/s、epoch time、DUT read bytes（应≥逻辑遍历量）、CPU。
- **判定：**AU≥90%；三次吞吐 CV≤5%；`--o-direct` 实际产生物理读 → Pass。`--o-direct` 未生效或数据集小于 datasize 要求 → **Invalid**。

### A3. Training 小文件读：RetinaNet/B200（对应 AI-TRN-004）

- 百万级 JPEG 小文件（352 GiB 全量），当前盘建议 reduced（如 10 万文件子集），验证 files/s 与 P99。
- **判定：**AU≥85%；无读取错误 → Pass；数据不足 → **Invalid**。

### A4. Checkpoint 突发写/读：Llama3-70B subset 8-rank（对应 AI-CKP-002）

- **命令**（写阶段；`--num-checkpoints-read 0`）：

```
mlpstorage open checkpointing run file `
  --model llama3-70b --systemname ai-ssd-win --num-processes 8 `
  --client-host-memory-in-gb 53 `
  --checkpoint-folder <DUT_DIR> --results-dir <RESULT_DIR> `
  --num-checkpoints-read 0
```

读阶段用 `--num-checkpoints-write 0`（写前清缓存 / 重启，Windows 无 drop\_caches）。

- **判定：**10 Save + 10 Load 全部完成；0 校验错误；最慢 rank 时间与最小 rank 吞吐达标 → Pass。缺 rank 或字节数不符 → **Invalid**。
- **容量提示：**8B full 105 GB×10 份 > 1 TB，当前 1 TB 盘只跑 smoke（1–2 轮）或换大容量 DUT。

### A5–A7. KV MLPerf Option 1/2/3（对应 AI-KV-001/002/003）

- **前置：**`--cache-dir` 映射到 DUT；每 option 三 trials。

```
Set-Location <REPO_ROOT>\kv_cache_benchmark
```

```
# Option 1: 8B NVMe-only, 200 users, GPU/CPU tier = 0
```

```
python -m kv_cache.cli --model llama3.1-8b --num-users 200 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> `
  --generation-mode none --performance-profile latency `
  --max-concurrent-allocs 16 --output <RESULT_DIR>\kv_opt1.json
```

```
# Option 2: 8B, CPU tier 4 GB 溢出到 NVMe, 100 users
```

```
python -m kv_cache.cli --model llama3.1-8b --num-users 100 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 4 --cache-dir <DUT_DIR> `
  --generation-mode none --performance-profile latency `
  --max-concurrent-allocs 16 --output <RESULT_DIR>\kv_opt2.json
```

```
# Option 3: 70B 大 KV 对象, 70 users, allocs=4
```

```
python -m kv_cache.cli --model llama3.1-70b-instruct --num-users 70 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> `
  --generation-mode none --performance-profile latency `
  --max-concurrent-allocs 4 --precondition --precondition-size-gb <SIZE_GB> `
  --output <RESULT_DIR>\kv_opt3.json
```

- **判定** (每 option) :

- `storage_entries > 0` 且 Tier-2 bytes > 0, 否则 **Invalid** (Option 2 若全部被 CPU tier 命中同样 Invalid) 。
- QoS: interactive profile 下 P95/P99/P99.9/P99.99 ≤ 50/100/150/200 ms ( `config.yaml` 已内置) 。
- 缺 rank/trial、OOM → Invalid/Fail。

## A8. VectorDB: 1M×1536 HNSW (对应 AI-VDB-002)

```
Set-Location <REPO_ROOT>\vdb_benchmark
```

```
vdbbench-modular --config vdbbench\benchmark\configs\1m_hnsw.yaml `
  --output-dir <RESULT_DIR>\vdb_1m_hnsw
```

- **判定**: row count=1M; Recall@10≥0.95 后比较 QPS/P99; load/index 阶段完成无错误。

## A9. VectorDB: 1M×1536 DISKANN (对应 AI-VDB-003)

```
vdbbench-modular --config vdbbench\benchmark\configs\1m_diskann.yaml `
  --io-trace-log <RESULT_DIR>\vdb_1m_diskann_trace.csv `
  --output-dir <RESULT_DIR>\vdb_1m_diskann
```

- 同时产出 trace 供轨道 B 使用。**判定**同 A8；外加 DUT 物理读字节显著>0（DISKANN 应命中磁盘）。

## 4. 轨道 B：模拟 Trace 测试（6 个 Case）

### B1. KV trace 生成：MLPerf Option 3 大对象（对应 AI-KV-021 前半）

```
Set-Location <REPO_ROOT>\kv_cache_benchmark
python -m kv_cache.cli --model llama3.1-70b-instruct --num-users 70 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> `
  --generation-mode none --performance-profile latency `
  --max-concurrent-allocs 4 --precondition --precondition-size-gb <SIZE_GB> `
  --io-trace-log <RESULT_DIR>\kv_opt3_trace.csv --output <RESULT_DIR>\kv_opt3_trace.json
```

- 验证**：CSV 行数>0、无时间倒退、Tier 列含 **Tier-2** 记录；否则 **Invalid**。

### B2. KV Tier-2 过滤 + Direct replay（对应 AI-KV-021）

- trace 必须先过滤，只保留 **Tier-2** 行（避免 GPU/CPU tier 稀释 SSD 负载）。

```
# 过滤（PowerShell / 仓库内任意脚本均可）
Import-Csv <RESULT_DIR>\kv_opt3_trace.csv | Where-Object { $_.Tier -eq 'Tier-2' } |
  Export-Csv <RESULT_DIR>\kv_opt3_tier2.csv -NoTypeInfoation

# 1x/2x/4x Direct I/O 回放（绕过页缓存，验证真实 SSD 能力）
Set-Location <REPO_ROOT>\vdb_benchmark
python -m vdbbench.replay <RESULT_DIR>\kv_opt3_tier2.csv `
  --data-dir <DUT_DIR>\kv_replay --direct-io --seed 42
python -m vdbbench.replay <RESULT_DIR>\kv_opt3_tier2.csv `
  --data-dir <DUT_DIR>\kv_replay --direct-io --speed 2 --seed 42
python -m vdbbench.replay <RESULT_DIR>\kv_opt3_tier2.csv `
  --data-dir <DUT_DIR>\kv_replay --direct-io --speed 4 --seed 42
```

- 判定**：P50/P95/P99 正常收敛；aligned bytes 与 trace 对象量一致；**--direct-io** 下 DUT 物理读>0 → Pass。回放缺对象/时间倒退 → **Invalid**。

### B3. KV prefill / decode 分离 trace (对应 AI-KV-015/016 的模拟形式)

```
# 写密集 prefill
python -m kv_cache.cli --model llama3.1-8b --num-users 50 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> --prefill-only `
  --io-trace-log <RESULT_DIR>\kv_prefill_trace.csv --output <RESULT_DIR>\kv_prefill.json

# 读密集 decode (先 precondition 预置 cache)
python -m kv_cache.cli --model llama3.1-8b --num-users 50 --duration 300 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> --decode-only `
  --precondition --precondition-size-gb <SIZE_GB> `
  --io-trace-log <RESULT_DIR>\kv_decode_trace.csv --output <RESULT_DIR>\kv_decode.json
```

- 分别 replay (同 B2) , **输出读写分离成绩**, 不用 mixed 平均值掩盖单方向瓶颈。

### B4. TP sweep 模拟 per-rank 对象大小 (对应 AI-KV-014 的 trace 形式)

```
# TP 1/2/4/8 各生成一条 trace, 观察对象大小随 TP 分片的变化
python -m kv_cache.cli --model llama3.1-70b-instruct --num-users 70 --duration 120 `
  --gpu-mem-gb 0 --cpu-mem-gb 0 --cache-dir <DUT_DIR> `
  --num-gpus 8 --tensor-parallel 4 --max-concurrent-allocs 4 `
  --io-trace-log <RESULT_DIR>\kv_70b_tp4_trace.csv --output <RESULT_DIR>\kv_70b_tp4.json
```

- 将各 TP 档位 trace 过滤 Tier-2 后 replay, 量化 per-rank 对象大小对 IOPS/BW/尾延迟的影响。

### B5. VDB trace replay (对应 AI-VDB-015)

- 用 A9 产出的 `vdb_1m_diskann_trace.csv` :

```
python -m vdbbench.replay <RESULT_DIR>\vdb_1m_diskann_trace.csv `
  --data-dir <DUT_DIR>\vdb_replay --direct-io
```

- **判定**: 逻辑 trace bytes 与 DUT 物理 bytes 分层报告; `--direct-io` 下物理读>0 → Pass; 口径混用 (逻辑当物理) → **Invalid**。

### B6. 大容量模型 trace 模拟 (对应 AI-KV-002 / 无法实跑的大模型)

- 用 `deepseek-v3` (MLA 压缩 KV)、`qwen3-32b` (256 KiB/token)、`gpt-oss-20b/120b` (MoE 小 KV) 生成 trace 并 replay, **不实际加载大模型**, 只模拟其 KV 对象分布。
- 用途: 在没有大容量 DUT / GPU 的前提下, 评估 SSD 对不同 KV 对象大小的能力曲线 (24–512 KiB/token) 。

## 5. 判定规则（统一）

| 规则            | 门槛                                                                                                                                                     |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 重复性           | 3 次吞吐 $CV \leq 5\%$                                                                                                                                    |
| 回归            | 同配置吞吐 $\geq$ golden 90%; P99 $\leq$ golden 110%                                                                                                        |
| Mixed 干扰      | 前台 P99 $\leq$ solo 1.5 $\times$ ; 后台吞吐 $\geq$ solo 80%                                                                                                 |
| KV QoS        | interactive P95/P99/P99.9/P99.99 $\leq$ 50/100/150/200 ms                                                                                              |
| VDB           | 先满足 Recall（建议 $\geq 0.95$ ）再比 QPS/P99                                                                                                                  |
| Invalid（一票否决） | <code>storage_entries=0</code> / Tier-2 bytes=0; 宣称 cold/direct 但 DUT 物理读 $\approx 0$ ; 缺 rank/trial; trace 时间倒退或对象缺失; 结果写入 DUT; 主机 CPU $>85\%$ 且磁盘未饱和 |

## 6. 监控（1 s 采样）

```
# Windows 1 秒 PhysicalDisk + 进程采样（结果归档为 windows_disk_1s.csv / process_1s.csv）
typeperf "\PhysicalDisk(*)\Disk Read Bytes/sec" "\PhysicalDisk(*)\Disk Write Bytes/sec" `
"\PhysicalDisk(*)\Avg. Disk sec/Read" "\PhysicalDisk(*)\Avg. Disk sec/Write" `
"\PhysicalDisk(*)\Current Disk Queue Length" -si 1 -sc <SECONDS> -o <RESULT_DIR>\windows_disk_1s.csv
```

- 关键用途：验证 workload **实际命中 DUT** (DET-PATH/DET-CACHE) ；发现 host-limited (DET-HOST) 与热降速 (DET-THERM) 。
- 所有时间戳统一 UTC；记录计数器实例到物理盘序列号的映射（Windows 计数器名可能本地化，通过系统索引解析）。

## 7. 结果归档

```
runs/<case_id>/<run_id>/
├─ manifest.json          # 配置、DUT、路径、commit、seed
├─ workload/              # summary.json / metadata.json / timeseries.json / stdout/stderr.log
├─ trace/                 # KV/VDB trace（轨道 B）
├─ windows_disk_1s.csv
├─ pre_post_health.json   # 温度、SMART 错误、可用空间（运行前后）
├─ artifacts.sha256
└─ verdict.json           # Pass/Fail/Invalid + 瓶颈归因
```

## 8. 执行顺序建议

| 批次      | Case                    | 前置                                          |
|---------|-------------------------|---------------------------------------------|
| 第一批（立即） | A1、A5–A7、A8、A9、B1、B2、B5 | 无（Milvus healthy、kv/vdb 已装）                 |
| 第二批     | A4、B3、B4、B6             | 容量检查（A4 smoke 1–2 轮）                        |
| 第三批     | A2、A3                   | <code>pip install -e ".[full]"</code> + MPI |

## 9. Definition of Done（简化版）

- 四家族各有  $\geq 1$  个实际测试 Case 出成绩，且每个成绩都证明命中 DUT（物理读 $>0$ ）。
- 轨道 B 至少完成一条完整链路：**生成 trace**  $\rightarrow$  **过滤 Tier-2**  $\rightarrow$  **Direct replay (1 $\times$ /2 $\times$ /4 $\times$ )**  $\rightarrow$  **输出 P50/P95/P99**。
- KV 同时报告 tier 命中与尾延迟；VDB 同时报告 Recall 与 QPS；Training 同时报告 AU 与 samples/s。
- 每个正式 Case  $\geq 3$  次重复，产物含配置、原始结果、1 s 监控、verdict。
- 容量不足的 Case（10 份 8B checkpoint、全量 405B/1T、983 GiB UNet3D）明确标 **Scaled**，不用缩小版冒充正式结果。